

越权漏洞研究报告

2025年12月12日

1. 绪论

1.1 越权漏洞概述

越权漏洞，作为Web应用安全领域中一种普遍且危害严重的逻辑漏洞，其核心在于应用程序未能对用户请求的访问权限进行有效验证，导致用户能够执行超出其授权范围的操作或访问未授权的数据资源。这种漏洞的本质是“失效的访问控制”（Broken Access Control），在OWASP（开放式Web应用程序安全项目）的十大安全风险中常年位居前列，凸显了其在现代Web安全中的重要性。根据权限提升或跨越的方向和性质，越权漏洞通常被划分为三种主要类型：**水平越权（Horizontal Privilege Escalation）**、**垂直越权（Vertical Privilege Escalation）**和**交叉越权（Cross Privilege Escalation）**。水平越权指的是攻击者通过修改请求中的身份标识参数，访问或操作与自己权限等级相同的其他用户的数据。垂直越权则是指低权限用户（如普通会员）通过某种手段，成功执行了高权限用户（如系统管理员）才能进行的操作。而交叉越权则是前两者的结合，攻击者不仅篡改了身份标识，还试图提升权限等级，是一种更为复杂的攻击模式。这些漏洞的产生，往往源于开发人员在实现业务逻辑时，对客户端提交的数据过分信任，忽略了在服务器端进行严格的权限校验，从而为攻击者留下了可乘之机。

1.2 越权漏洞的危害

越权漏洞的危害程度与其所影响业务功能的重要性直接相关，其潜在风险不容小觑。在水平越权场景中，攻击者可能通过遍历用户ID等参数，批量获取系统中所有同级别用户的敏感信息，如个人资料、订单详情、私信内容等，这实质上构成了一种变相的数据泄露（“脱裤”），严重侵犯了用户隐私。由于这类攻击请求与正常业务请求在形式上几乎没有区别，不包含明显的恶意特征，因此很难被传统的防火墙或入侵检测系统（IDS）所发现，具有极高的隐蔽性。在垂直越权场景中，危害则更为严重。一旦低权限用户成功获取了管理员权限，其能够造成的破坏将是灾难性的。例如，攻击者可以创建、删除或修改其他用户账户，篡改系统配置，上传恶意文件，甚至获取服务器控制权，导致整个应用乃至企业内网的安全防线被彻底攻破。交叉越权由于结合了两种类越权方式的特点，其攻击路径更加灵活多变，防御难度也相应增加，最终可能导致的危害同样是系统性的、毁灭性的。

1.3 报告结构

本报告旨在对越权漏洞进行全面、系统、深入的研究，特别是针对PHP编程语言环境下的漏洞原理、挖掘与防御。报告将遵循以下结构展开：

- **第二章：水平越权（Horizontal Privilege Escalation）**：本章将详细剖析水平越权的漏洞原理，包括其定义、成因以及在PHP应用中的常见场景。随后，将系统介绍针对水平越权的黑盒测试和白盒审计方法，并分享一系列实用的漏洞挖掘技巧。最后，本章将重点阐述有效的整改措施，并提供具体的PHP代码示例，演示如何从根本上修复此类漏洞。
- **第三章：垂直越权（Vertical Privilege Escalation）**：本章将聚焦于垂直越权，深入探讨其定义、成因和常见场景。同样地，本章将详细介绍垂直越权的挖掘方法和技巧，并重点讲解基于角色的访问控制（RBAC）等先进的权限管理模型，以及如何利用中间件或拦截器实现统一的权限校验。最后，将通过PHP代码示例展示修复垂直越权的具体实现。
- **第四章：交叉越权（Hybrid Privilege Escalation）**：作为水平与垂直越权的结合体，本章将首先明确交叉越权的定义和原理，并分析其与另外两种越权类型的内在联系。接着，将探讨如何综合运用前两章的挖掘方法来发现交叉越权漏洞，并提出一套综合性的修复策略，辅以PHP代码示例进行说明。
- **第五章：结论与建议**：在报告的最后，将对全文进行总结，提炼出核心观点。同时，从安全开发的角度，为开发人员提供一系列预防越权漏洞的最佳实践建议。最后，将对未来越权漏洞的研究方向进行展望。

通过上述结构，本报告力求为读者呈现一幅关于越权漏洞的全景图，不仅帮助安全研究人员和开发人员理解其内在机理，更能提供一套行之有效的实战指南。

2. 水平越权（Horizontal Privilege Escalation）

2.1 漏洞原理

2.1.1 定义与成因

水平越权（Horizontal Privilege Escalation）是一种访问控制失效的漏洞，其核心特征是攻击者在不改变自身用户权限等级（Role）的前提下，通过篡改请求中的身份标识（ID），成功访问或操作了其他具有相同权限等级的用户所拥有的资源。这种漏洞的本质在于应用程序未能对“资源归属”进行严格的校验。在一个设计良好的系统中，每个用户应该只能访问和操作自己的数据，例如，用户A只能查看和修改自己的个人资料，而不能访问用户B的资料。然而，当系统在处理数据请求时，仅仅依赖于客户端提供的、可被轻易篡改的参数（如URL中的 `user_id` 或表单中的 `username`）来定位资源，而没有在服务器端验证当前登录用户是否对该资源拥有合法的所有权或操作权限，水平越权漏洞便应运而生。

其根本成因可以归结为以下几点：

1. **过度信任客户端数据**：开发人员错误地认为从客户端（如浏览器、APP）传来的数据是可信的，没有意识到这些数据可以被用户或攻击者轻易修改。

2. **缺失资源归属校验**：在执行业务逻辑（如查询、更新、删除）之前，代码逻辑中没有包含“当前登录用户是否为所操作资源的拥有者”这一关键判断。
3. **不安全的直接对象引用（IDOR）**：这是水平越权最常见的表现形式。系统直接使用用户提供的、未经校验的标识符（如数据库主键、文件名）来访问内部对象，攻击者通过修改这个标识符就能访问到其他对象。

2.1.2 PHP中的常见场景

在基于PHP开发的Web应用中，水平越权漏洞经常出现在以下典型场景中：

- **用户信息查看与修改**：这是最常见的场景。例如，一个查看个人资料的页面，其URL可能形如 `profile.php?user_id=123`。如果后台代码直接根据 `$_GET['user_id']` 的值从数据库中查询并返回数据，而没有检查 `$_SESSION['user_id']` 是否与 `$_GET['user_id']` 一致，那么攻击者只需将URL中的 `123` 修改为其他用户的ID，如 `124`，就能查看到他人的资料。
- **订单详情查询**：在电商系统中，用户可以通过订单号查看自己的订单。如果订单查询接口 `order.php?order_id=ORD20251212001` 没有验证该订单是否属于当前登录用户，攻击者就可以通过遍历订单号来查看其他用户的订单信息，包括商品、价格、收货地址等敏感数据。
- **文件下载与访问**：当系统提供文件下载功能时，如果下载链接 `download.php?file_id=5` 直接映射到服务器上的文件，而没有检查当前用户是否有权限访问该文件，攻击者就可能通过修改 `file_id` 下载到其他用户的私密文件。
- **密码重置功能**：某些密码重置逻辑可能存在缺陷。例如，系统在发送重置邮件后，在重置页面通过隐藏字段或URL参数传递用户标识，如果攻击者能够捕获并修改这个标识，就可能重置其他用户的密码。

以下是一个典型的存在水平越权漏洞的PHP代码片段：

php

复制

```
<?php
// 假设这是从URL参数中获取的用户ID
$targetUserId = $_GET['user_id'];

// 存在漏洞的查询：直接根据$targetUserId查询数据库，未进行任何权限验证
$sql = "SELECT username, email, phone FROM users WHERE id = $targetUserId";
$result = mysqli_query($conn, $sql);
```

```
if ($row = mysqli_fetch_assoc($result)) {  
    echo "Username: " . $row['username'] . "<br>";  
    echo "Email: " . $row['email'] . "<br>";  
    echo "Phone: " . $row['phone'];  
} else {  
    echo "User not found.";  
}  
?>
```

在这段代码中，无论当前登录的用户是谁，只要他知道其他用户的ID，就可以通过修改 `user_id` 参数来查看任何人的信息，这就是一个典型的水平越权漏洞。

2.2 漏洞挖掘方法

2.2.1 黑盒测试方法

黑盒测试是在不了解应用程序内部代码和架构的情况下，从外部模拟用户行为来发现漏洞的方法。针对水平越权，黑盒测试主要遵循以下步骤：

- 1. 识别敏感参数：**首先，需要全面浏览应用程序的功能，特别是那些涉及用户个人数据操作的模块。使用浏览器开发者工具或代理工具（如Burp Suite）捕获所有HTTP请求（GET、POST、PUT、DELETE等），并仔细分析请求中的参数。重点关注那些看起来像用户标识符的参数，例如 `id` 、 `user_id` 、 `uid` 、 `username` 、 `account_id` 、 `order_id` 等。这些参数是水平越权攻击的潜在切入点。
- 2. 创建测试账户：**为了有效地进行测试，至少需要创建两个或更多的普通用户账户。使用账户A登录系统，并访问一个会返回个人敏感信息的功能，例如“我的订单”或“个人中心”。此时，记录下该请求的完整信息，包括URL、请求方法、请求头和请求体。
- 3. 参数替换与重放：**在Burp Suite的“Repeater”或“Intruder”模块中，将捕获到的请求进行修改。将请求中代表用户身份或资源的参数值，从账户A的标识符替换为账户B的标识符。例如，如果请求是 `GET /api/user/123/profile` ，则将 `123` 替换为账户B的ID `456` 。然后，使用账户A的会话（即带着账户A的Cookie）发送这个被修改过的请求。
- 4. 分析响应结果：**观察服务器的响应。如果服务器返回了账户B的个人数据（例如，用户名、邮箱、订单详情等），而不是返回“权限不足”或“未找到数据”的错误信息，那么就成功地发现了一个水平越权漏洞。这意味着服务器端没有验证请求者（账户A）是否对被请求的资源（账户B的数据）拥有访问权限。

2.2.2 白盒代码审计

白盒代码审计是通过直接分析应用程序的源代码来发现安全漏洞的方法。对于水平越权，白盒审计更为精准和高效，能够从根本上定位问题所在。

1. **追踪用户输入：**首先，需要定位所有接收用户输入的代码点。在PHP中，这通常涉及全局变量，如 `$_GET`、`$_POST`、`$_REQUEST`、`$_COOKIE` 和 `$_SERVER`。使用代码搜索工具（如 `grep` 或IDE的搜索功能）查找这些变量的使用位置。特别关注那些将用户输入直接用于数据库查询、文件操作或逻辑判断的地方。
2. **分析数据流：**一旦找到用户输入的入口，就需要追踪这些数据在程序中的流向。观察这些输入数据是否被直接用于构造SQL查询语句，例如 `"SELECT * FROM users WHERE id = " . $_GET['id']`。如果存在这种情况，就需要高度警惕。
3. **检查权限验证逻辑：**在数据流的关键节点，特别是即将执行敏感操作（如查询、更新、删除）之前，检查是否存在有效的权限验证逻辑。理想的权限验证应该基于会话（Session）中存储的当前登录用户信息。例如，代码应该包含类似 `if ($_SESSION['user_id'] == $_GET['user_id'])` 的判断。如果缺少这样的判断，或者判断逻辑存在缺陷（如可以被轻易绕过），则水平越权漏洞存在。
4. **审查ORM和框架使用：**如果应用使用了对象关系映射（ORM）框架（如Laravel的Eloquent或Doctrine），需要检查其查询构建器的使用方式。确保在查询时，总是附加了当前用户的约束条件。例如，查询订单时，应该使用 `Order::where('user_id', Auth::id())->find($orderId)`，而不是直接使用 `Order::find($orderId)`。

2.3 漏洞挖掘技巧

2.3.1 识别关键参数

在水平越权漏洞的挖掘过程中，快速而准确地识别出请求中用于控制身份或资源的关键参数，是整个测试流程的核心和起点。这些参数是攻击者实施越权攻击的“钥匙”，因此，测试人员需要具备敏锐的洞察力，从复杂的HTTP请求中定位这些关键字段。

关键参数通常具有以下几个特征：

1. **明显的命名特征：**参数名往往直接暴露了其作用。最常见的如 `id`，`user_id`，`uid`，`username`，`account`，`order_id`，`file_id`，`msg_id` 等。这些参数名直观地表明了它们用于标识一个特定的用户或数据对象。在抓包时，应优先关注这些具有明确意义的参数。
2. **数值型或规律性字符串：**参数的值通常是整数（如 `123`，`456`）或具有明显规律的字符串（如 `user_001`，`order_20231201_001`）。这种可预测、可枚举的特性使得攻击

者可以通过简单地递增或修改这些值来尝试访问其他资源。例如，将 `user_id=100` 修改为 `user_id=101`，就可能访问到下一个用户的数据。

3. **出现在敏感操作的请求中**：关键参数往往出现在那些涉及数据增、删、改、查的请求中。例如，修改密码、查看订单详情、删除个人文件、编辑个人资料等操作。在进行黑盒测试时，应重点抓取这些敏感操作的数据包进行分析。
4. **可能隐藏在非直观位置**：除了URL查询字符串和POST请求体，关键参数还可能出现在其他不那么直观的位置，需要测试人员特别留意：
 - **Cookie**：某些应用会将用户ID或会话标识存储在Cookie中。例如，一个名为 `user_info` 的Cookie，其值可能是经过编码的用户ID。攻击者可以尝试解码（如Base64解码）并修改这个值。
 - **HTTP请求头**：自定义的请求头中也可能包含身份标识，如 `X-User-Id: 123`。
 - **JSON/XML请求体**：在AJAX请求中，身份标识可能作为JSON对象的一个属性，如 `{"user_id": "123"}`。
5. **经过编码或加密**：为了增加安全性，一些应用会对关键参数进行编码（如Base64）或加密。虽然这增加了直接修改的难度，但并非不可逾越。测试人员可以尝试识别编码方式（Base64编码的特征是末尾常有 `=` 或 `==`），并进行解码。对于加密的情况，如果加密算法或密钥被破解，同样可以伪造参数。即使无法破解，也可以尝试重放被加密的数据包，观察服务器的行为。

通过综合运用这些识别技巧，测试人员可以更高效地从海量网络流量中筛选出潜在的水平越权攻击点，为后续的漏洞利用和验证打下坚实的基础。

2.3.2 利用BurpSuite等工具

BurpSuite是一款功能强大的Web应用安全测试工具集，它为挖掘水平越权漏洞提供了极大的便利。通过其代理（Proxy）、重放器（Repeater）、入侵者（Intruder）等模块，测试人员可以高效地拦截、分析、修改和自动化测试HTTP请求，从而系统性地发现和验证越权漏洞。

以下是利用BurpSuite挖掘水平越权漏洞的典型流程和技巧：

1. **设置代理并拦截流量**：首先，需要将浏览器的代理设置为BurpSuite的监听地址（默认为127.0.0.1:8080）。然后，在BurpSuite的“Proxy” -> “Intercept”选项卡中开启拦截功能。此时，浏览器发出的所有HTTP请求都会被BurpSuite截获，允许测试人员在请求发送到服务器之前进行查看和修改。
2. **分析请求并定位参数**：当一个请求被拦截后，可以在BurpSuite中查看其详细信息，包括URL、请求头、请求体等。利用上节提到的识别技巧，定位可能存在水平越权风险的关键

参数。例如，在一个查看用户信息的GET请求中，发现URL为 `/user/info?uid=1001` 。

3. **使用Repeater进行手动测试：**对于初步怀疑存在漏洞的请求，可以将其发送到“Repeater”模块。在Repeater中，可以方便地修改请求的任何部分，并反复发送给服务器以观察响应。例如，将 `uid=1001` 修改为 `uid=1002`，然后点击“Send”。如果服务器返回了用户1002的信息，则漏洞得到验证。Repeater模块非常适合进行精细化的手动测试和调试。
4. **使用Intruder进行自动化爆破：**当需要测试大量可能的参数值时（例如，遍历用户ID），手动修改效率低下。此时，可以使用“Intruder”模块进行自动化攻击。将可疑参数的位置用“\$”符号标记，然后在“Payloads”选项卡中设置载荷类型（如数字范围、字典文件等）。Intruder会自动用载荷替换标记的位置，并发送大量请求，然后分析每个请求的响应，帮助测试人员快速发现哪些参数值是有效的，从而判断是否存在越权。例如，可以设置Payload为从1到1000的数字，来测试 `uid` 参数。
5. **利用Comparer对比响应差异：**在测试过程中，可能会遇到修改参数后，页面内容看起来没有变化，但实际上响应内容已经不同的情况。BurpSuite的“Comparer”模块可以帮助对比两个HTTP响应的异同。通过对比正常请求的响应和修改参数后的响应，可以发现隐藏在页面背后的敏感信息泄露。
6. **使用Scanner进行辅助扫描：**BurpSuite的专业版包含一个“Scanner”模块，它可以对拦截到的请求进行自动化的漏洞扫描。虽然Scanner在发现复杂的逻辑漏洞方面能力有限，但它可以帮助识别一些常见的安全问题，如未授权访问等，为手动测试提供线索。

通过熟练运用BurpSuite的这些功能，测试人员可以系统、高效地进行水平越权漏洞的挖掘，从拦截流量到定位参数，再到手动验证和自动化爆破，形成一个完整的测试闭环。

2.4 整改措施

2.4.1 后端权限验证

修复水平越权漏洞最根本、最核心的措施是在服务器端实施严格的权限验证。这意味着，在处理任何涉及用户私有数据的请求时，应用程序必须验证当前登录用户是否有权限访问或操作请求中所指定的数据对象。仅仅依赖前端的隐藏或禁用是不可靠的，因为攻击者可以直接绕过前端，向后端发送伪造的请求。

一个健壮的后端权限验证机制应遵循以下原则：

1. **基于所有权的验证：**核心思想是“你只能操作你自己的数据”。在处理请求时，系统需要从会话（如 `$_SESSION`）中获取当前登录用户的唯一标识（如 `user_id`），然后将其

与请求中要操作的数据对象的所有者标识进行比较。如果两者不匹配，则应立即拒绝该请求，并返回一个通用的错误信息（如“权限不足”或“未找到资源”），避免泄露过多信息。

php

复制

```
// 安全的代码示例
session_start();
$current_user_id = $_SESSION['user_id']; // 从会话中获取当前用户ID

$target_user_id = $_GET['id']; // 获取请求中要查看的用户ID

if ($current_user_id != $target_user_id) {
    // 权限验证失败
    http_response_code(403);
    die('Access Denied');
}

// 验证通过，继续执行业务逻辑
$sql = "SELECT * FROM users WHERE id = ?";
// ... 执行查询
```

2. **在数据访问层进行验证**：权限验证的逻辑应该尽可能地靠近数据访问层。这意味着，在执行数据库查询或更新操作之前，就应该完成权限检查。这样可以确保无论请求来自哪个入口（Web页面、API接口、后台任务等），权限控制都是一致的。例如，可以将权限验证封装成一个独立的函数或中间件，在所有需要的地方调用。
3. **不要信任任何来自客户端的标识**：永远不要直接使用 `$_GET`，`$_POST`，`$_COOKIE` 等超全局变量中的值作为身份或权限判断的唯一依据。这些值都可以被攻击者轻易伪造。当前用户的身份信息应该只从服务器端可控的会话（Session）或经过严格验证的JWT（JSON Web Token）中获取。
4. **统一权限管理**：对于复杂的系统，建议建立一套统一的权限管理模型，如基于角色的访问控制（RBAC）或基于属性的访问控制（ABAC）。虽然RBAC更多地用于解决垂直越权问题，但其核心思想——将权限与身份解耦，进行集中管理——同样适用于水平越权的精细化控制。例如，可以定义“资源所有者”这一角色，并将其与特定的数据对象关联。

通过在服务端强制执行基于所有权的权限验证，可以从根本上杜绝水平越权漏洞，确保每个用户的数据边界清晰、不可侵犯。

2.4.2 数据关联查询

除了直接在业务逻辑中进行权限验证外，另一种更优雅、更安全的修复水平越权漏洞的方法是使用数据关联查询。这种方法的核心思想是，在查询数据库时，直接将当前用户的身份作为查询条件之一，让数据库本身来过滤掉不属于当前用户的数据。这样，即使攻击者尝试修改请求参数，也无法查询到他人的数据，因为查询语句本身就限制了数据范围。

这种方法的优势在于：

1. **安全性更高**：权限控制逻辑被内嵌在SQL查询语句中，减少了在应用层因逻辑疏忽而导致漏洞的可能性。数据库会严格地只返回满足所有条件的数据。
2. **代码更简洁**：避免了在应用层编写大量的 `if...else...` 权限判断语句，使得业务逻辑代码更加清晰、易读。
3. **性能更优**：数据库引擎在查询时进行权限过滤，通常比在应用层获取数据后再进行判断要高效。

以下是使用数据关联查询修复水平越权的PHP代码示例：

场景：删除用户的收款方式

假设存在漏洞的代码直接根据收款方式的ID进行删除：

php

📄 复制

```
// 存在漏洞的代码
$payment_id = $_POST['payment_id'];
$sql = "DELETE FROM payments WHERE id = $payment_id";
// 执行SQL，未验证该payment_id是否属于当前用户
```

修复方法一：在WHERE子句中加入用户ID条件

php

📄 复制

```
// 安全的代码示例
session_start();
$current_user_id = $_SESSION['user_id']; // 获取当前用户ID
$payment_id = $_POST['payment_id'];

// 使用预处理语句防止SQL注入，并加入用户ID作为查询条件
$sql = "DELETE FROM payments WHERE id = ? AND user_id = ?";
$stmt = $pdo->prepare($sql);
$stmt->execute([$payment_id, $current_user_id]);
```

```
// 检查是否真的有行被删除
if ($stmt->rowCount() > 0) {
    echo "Payment deleted successfully.";
} else {
    // 如果没有行被删除, 说明该支付方式不属于当前用户, 或者ID不存在
    echo "Payment not found or access denied.";
}
```

在这个例子中, `user_id = ?` 这个条件确保了只有属于当前用户的支付方式才会被删除。如果攻击者尝试删除他人的支付方式, 由于 `user_id` 不匹配, `DELETE` 语句将不会影响到任何行。

修复方法二: 利用ORM框架的关联关系 (以Laravel为例)

如果使用Laravel等现代PHP框架, 可以利用其强大的ORM (对象关系映射) 功能来更优雅地实现数据关联查询。

php

复制

```
// 在User模型中定义与Payment的关联
class User extends Model {
    public function payments() {
        return $this->hasMany(Payment::class);
    }
}

// 在控制器中, 通过关联关系进行删除
class PaymentController extends Controller {
    public function destroy($id) {
        // 获取当前登录用户
        $user = auth()->user();

        // 通过关联关系查询并删除, 确保只删除属于当前用户的记录
        $deleted = $user->payments()->where('id', $id)->delete();

        if ($deleted) {
            return response()->json(['message' => 'Deleted successfully']);
        } else {
            return response()->json(['message' => 'Payment not found'], 404);
        }
    }
}
```

这种方法利用了框架的模型关联，代码更加语义化。 `$user->payments()` 这个调用本身就限制了查询范围只在当前用户的支付方式集合内，从而天然地防止了水平越权。

无论是使用原生SQL还是ORM框架，数据关联查询都是一种非常有效且推荐的修复水平越权漏洞的方法。它将权限控制下沉到数据访问层，使得安全策略与业务逻辑结合得更紧密，从而构建出更安全的应用。

2.4.3 代码示例：修复水平越权

为了更直观地展示水平越权漏洞的修复过程，本节将提供一个完整的PHP代码示例，对比修复前后的代码逻辑。该示例模拟一个用户查看个人资料的场景。

场景描述：

一个Web应用提供了用户个人资料页面，用户登录后可以通过访问 `profile.php?username=xxx` 来查看自己的资料。其中 `username` 是用户的唯一标识。

存在漏洞的代码 (`profile_vulnerable.php`)：

php

复制

```
<?php
// 模拟数据库连接
$db = new PDO('mysql:host=localhost;dbname=test', 'root',
'password');

// 直接从URL 参数中获取用户名，未进行任何权限验证
$username = $_GET['username'];

// 根据用户名查询用户信息
$stmt = $db->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$username]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if ($user) {
    echo "<h1>User Profile</h1>";
    echo "<p>Username: " . htmlspecialchars($user['username']) . "
</p>";
    echo "<p>Email: " . htmlspecialchars($user['email']) . "</p>";
    echo "<p>Phone: " . htmlspecialchars($user['phone']) . "</p>";
} else {
    echo "User not found.";
}
?>
```

漏洞分析：

在上述代码中，程序直接从 `$_GET['username']` 获取要查询的用户名，并直接用于SQL查询。它完全没有检查当前登录的用户是谁，也没有验证当前用户是否有权限查看

`$_GET['username']` 指定的用户资料。因此，任何用户（包括未登录的访客）都可以通过修改URL中的 `username` 参数来查看任意用户的资料，这是一个典型的水平越权漏洞。

修复后的代码 (profile_secure.php)：

php

复制

```
<?php
// 启动会话
session_start();

// 模拟数据库连接
$db = new PDO('mysql:host=localhost;dbname=test', 'root',
'password');

// 检查用户是否已登录
if (!isset($_SESSION['loggedin']) || $_SESSION['loggedin'] !== true)
{
    die("Error: You are not logged in. Please <a
href='login.php'>login</a> first.");
}

// 从会话中获取当前登录用户的用户名
$current_username = $_SESSION['username'];

// 从URL参数中获取要查看的用户名
$target_username = $_GET['username'];

// 【核心修复】进行权限验证：检查当前用户是否正在尝试查看自己的资料
if ($current_username !== $target_username) {
    // 权限验证失败，拒绝访问
    http_response_code(403);
    die("Access Denied: You can only view your own profile.");
}

// 权限验证通过，安全地查询数据库
$stmt = $db->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$target_username]);
$user = $stmt->fetch(PDO::FETCH_ASSOC);

if ($user) {
```

```
    echo "<h1>Your Profile</h1>";
    echo "<p>Username: " . htmlspecialchars($user['username']) . "
</p>";
    echo "<p>Email: " . htmlspecialchars($user['email']) . "</p>";
    echo "<p>Phone: " . htmlspecialchars($user['phone']) . "</p>";
} else {
    echo "User not found.";
}
?>
```

修复分析：

修复后的代码引入了会话管理，并从会话中获取当前登录用户的用户名

`$current_username`。在查询数据库之前，它增加了一个关键的权限验证步骤：比较 `$current_username` 和 `$target_username`。只有当两者相等时，查询才会继续执行。如果用户尝试查看他人的资料，程序会返回一个403 Forbidden错误，从而有效地阻止了水平越权攻击。

这个简单的示例清晰地展示了，通过在服务端增加一道基于所有权的权限验证关卡，可以完全修复水平越权漏洞。这个原则适用于所有涉及用户私有数据的操作。

3. 垂直越权 (Vertical Privilege Escalation)

3.1 漏洞原理

3.1.1 定义与成因

垂直越权 (Vertical Privilege Escalation)，又称权限提升，是一种更为危险的访问控制漏洞。它指的是一个**低权限用户**（例如，普通注册用户）通过利用应用程序的缺陷，成功执行了本应只有**高权限用户**（例如，系统管理员、超级管理员）才能执行的操作或访问其专属资源。与水平越权不同，垂直越权跨越了权限的“等级”界限，实现了从低级别到高级别的权限跃迁。这种漏洞的成因主要在于应用程序在实现功能访问控制时存在逻辑缺陷或完全缺失。

其根本成因可以归结为以下几点：

- 缺失角色权限校验：**这是最常见的原因。在需要管理员权限才能访问的页面或接口中，后台代码没有检查当前登录用户的角色 (Role) 或权限级别 (Permission Level)。例如，一个管理员后台页面 `admin.php` 可能直接根据用户是否登录来判断访问权限，而没有验证其角色是否为 `admin`。
- 不安全的权限标识：**有时，应用程序会将用户的权限信息（如 `role=admin`）存储在 Cookie、隐藏的表单字段或 URL 参数中。攻击者可以通过修改这些客户端数据来伪造高权限身份。

3. **功能暴露与未授权访问**：某些高权限功能（如用户管理、系统配置）的接口或页面URL可能被意外暴露。如果攻击者能够直接猜测或通过目录扫描发现这些URL，并且系统没有进行有效的权限拦截，攻击者就能直接访问。
4. **弱权限验证逻辑**：权限验证逻辑可能存在漏洞，例如，验证条件可以被轻易绕过，或者验证只在前端进行，后端没有二次确认。

3.1.2 PHP中的常见场景

在PHP应用中，垂直越权漏洞常常出现在以下场景：

- **后台管理页面**：这是垂直越权的重灾区。许多系统在开发初期，为了图方便，可能只对后台登录做了简单的认证，而没有对每个管理功能（如添加用户、删除文章、修改系统设置）进行细粒度的权限校验。一个普通用户登录后，如果直接访问 `/admin/` 目录下的某个管理页面，可能会因为缺少权限检查而成功进入。
- **功能菜单与按钮的显示逻辑**：前端页面可能会根据用户角色动态显示菜单和按钮。例如，普通用户看不到“用户管理”菜单。但如果后端API接口没有相应的权限验证，攻击者可以通过直接调用该接口（例如，通过Burp Suite发送 `POST /api/admin/create_user` 请求）来执行管理员功能。
- **文件上传功能**：某些文件上传功能可能只允许管理员使用，但如果上传处理脚本 `upload.php` 没有验证上传者的权限，普通用户也可能利用该功能上传恶意文件，进而可能导致远程代码执行。
- **API接口调用**：在前后端分离的架构中，后端API接口是独立的。如果API接口的设计没有遵循“无状态”和“每次请求都需验证”的原则，攻击者就可能通过伪造请求来调用高权限的API。

以下是一个典型的垂直越权漏洞的PHP代码片段：

php

复制

```
<?php
// admin_dashboard.php - 管理员后台页面

// 只检查了用户是否登录，但没有检查其角色
if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {
    header("Location: login.php");
    exit();
}
```

```
// 漏洞点：没有验证当前用户是否是管理员
// 任何已登录的用户都可以访问此页面并执行管理员功能

echo "<h1>Admin Dashboard</h1>";
echo "<p>Welcome, " . $_SESSION['username'] . "!</p>";

// 假设这里有一些管理员功能，如删除用户
if (isset($_GET['action']) && $_GET['action'] == 'delete_user') {
    $user_id_to_delete = $_GET['user_id'];
    // 直接执行删除操作，没有权限检查
    $sql = "DELETE FROM users WHERE id = $user_id_to_delete";
    mysqli_query($conn, $sql);
    echo "<p>User $user_id_to_delete has been deleted.</p>";
}

// ... 其他管理员功能
?>
```

在这段代码中，虽然检查了用户是否登录，但完全没有验证其角色。这意味着，一个普通的、已经登录的用户，只要在浏览器中访问 `admin_dashboard.php?action=delete_user&user_id=123`，就能执行删除用户的操作，这就是一个典型的垂直越权漏洞。

只要在浏览器中访问 `admin_dashboard.php?action=delete_user&user_id=123`，就能执行删除用户的操作，这就是一个典型的垂直越权漏洞。

3.2 漏洞挖掘方法

3.2.1 黑盒测试方法

针对垂直越权的黑盒测试，核心思路是“以低权限之身，行高权限之事”。

- 枚举高权限路径：**首先，需要对应用程序的URL结构进行分析。通常，管理员相关的页面或接口会包含特定的关键词，如 `admin`、`manage`、`console`、`backend` 等。可以尝试手动输入这些URL（例如，`/admin/`，`/manage/users.php`），看是否能直接访问。同时，可以使用目录扫描工具（如DirBuster、御剑）来发现隐藏的管理后台入口。
- 角色切换测试：**准备两个账户，一个普通用户账户和一个管理员账户。首先，用管理员账户登录，并执行一个典型的管理操作（例如，创建一个新用户），通过Burp Suite捕获这个操作的完整HTTP请求。然后，退出管理员账户，用普通用户账户登录。在Burp Suite中，找到之前捕获的管理员请求，将其发送到Repeater。在Repeater中，将请求头中的 `Cookie` 字段替换为普通用户的Cookie，然后发送请求。如果服务器返回了成功的响应（例如，显示“用户创建成功”），则表明存在垂直越权漏洞。
- 参数与Cookie分析：**仔细分析登录成功后的响应包，特别是Set-Cookie头。有时，权限信息（如 `role=admin`）可能会被直接设置在Cookie中。尝试修改Cookie中的这些值，

然后访问高权限功能。此外，也要关注POST请求的请求体，看是否有类似 `role=user` 的参数可以被修改。

3.2.2 白盒代码审计

白盒审计能够更精确地定位垂直越权的根源。

1. **定位权限检查点**：在代码库中搜索与权限检查相关的关键词，如 `role`、`permission`、`admin`、`privilege`、`check` 等。找到所有执行权限判断的函数或代码块。
2. **分析权限验证逻辑**：仔细检查这些权限检查点的逻辑是否严谨。例如，是否存在 `if ($user['role'] == 'admin')` 这样的判断。如果存在，需要进一步分析这个判断是否在所有的敏感操作前都被调用。特别要注意那些通过 `include` 或 `require` 引入的文件，确保这些文件中的权限检查也被正确执行。
3. **检查访问控制列表（ACL）或中间件**：如果应用使用了框架，应该检查其访问控制中间件或路由守卫的实现。例如，在Laravel中，可以检查路由是否被 `>middleware('can:admin')` 或 `->middleware('auth:admin')` 所保护。确保所有需要管理员权限的路由都被正确地应用了这些中间件。
4. **寻找“漏网之鱼”**：有些功能可能是在开发后期添加的，开发者可能会忘记为其添加权限验证。因此，需要系统地检查所有控制器（Controller）和独立的功能脚本，确保每个可能执行敏感操作的函数或脚本都包含了必要的权限检查。

3.3 漏洞挖掘技巧

3.3.1 重放攻击

重放攻击是挖掘垂直越权漏洞的经典技巧。其核心在于捕获一个高权限用户的合法请求，然后在低权限用户的会话中重新发送这个请求。

操作步骤：

1. **捕获高权限请求**：使用Burp Suite作为代理，用管理员账户登录系统，并执行一个典型的管理操作，例如删除一个用户。在Burp Suite的“Proxy” -> “HTTP history”中找到这个 `DELETE /api/users/123` 的请求。
2. **切换到普通用户**：在浏览器中注销管理员账户，然后登录一个普通用户账户。
3. **重放请求**：在Burp Suite的HTTP历史记录中，右键点击之前捕获的管理员请求，选择“Send to Repeater”。在“Repeater”标签页中，找到请求头中的 `Cookie` 字段。将其值替换为当前普通用户的Cookie值（可以从浏览器开发者工具的“Application” ->

“Cookies”中复制，或者直接在Burp Suite的HTTP历史中找到一个普通用户的请求并复制其Cookie）。

4. **发送并观察：**点击“Go”按钮发送修改后的请求。观察响应。如果服务器返回 200 OK 并确认删除成功，或者返回了被删除用户的详细信息，那么就成功发现了一个垂直越权漏洞。如果服务器返回 403 Forbidden 或 401 Unauthorized ，则说明权限控制是有效的。

3.3.2 修改身份标识

这种方法侧重于修改请求中可能存在的、代表用户身份或权限的参数。

操作步骤：

1. **分析请求：**使用普通用户登录，并访问一个普通功能。用Burp Suite捕获请求，仔细分析请求的所有部分：URL路径、URL参数、POST请求体、Cookie、HTTP头等。
2. **寻找可疑参数：**寻找任何可能与权限相关的参数。例如，在POST请求体中看到 `role=user` ，或者在Cookie中看到 `is_admin=0` 。
3. **进行修改：**在Burp Suite的Repeater中，尝试修改这些参数。将 `role=user` 改为 `role=admin` ，将 `is_admin=0` 改为 `is_admin=1` 。
4. **测试高权限功能：**修改后，尝试访问一个需要管理员权限的功能，或者直接发送一个管理员的API请求。观察结果。如果权限被成功提升，则漏洞存在。
5. **组合攻击：**有时，单一的修改可能无效。可以尝试组合多种修改，例如，同时修改Cookie中的 `role` 参数和URL中的 `user_id` 参数，进行更复杂的攻击尝试。

3.4 整改措施

3.4.1 RBAC权限控制

基于角色的访问控制（Role-Based Access Control, RBAC）是解决垂直越权问题的根本方案。RBAC的核心思想是将权限与角色相关联，再将角色分配给用户，从而实现用户与权限的逻辑分离。这种模型使得权限管理更加灵活、清晰和易于维护。

一个典型的RBAC模型通常包含以下几个核心实体：

- **用户（User）**：系统的使用者。
- **角色（Role）**：一组权限的集合，代表了一类用户在系统中的职责，如“普通用户”、“编辑”、“管理员”。

- **权限 (Permission)** : 对系统资源的访问许可, 通常表现为对某个功能 (如 `create_post`) 或数据 (如 `view_all_users`) 的操作。
- **会话 (Session)** : 用户与系统交互的上下文, 用户的角色和权限信息通常在登录后存储在会话中。

在PHP中实现RBAC, 通常需要设计如下几张数据库表:

- `users` (用户表)
- `roles` (角色表)
- `permissions` (权限表)
- `user_roles` (用户-角色关联表, 多对多关系)
- `role_permissions` (角色-权限关联表, 多对多关系)

当用户登录时, 系统查询数据库, 获取该用户所拥有的所有角色, 以及这些角色所拥有的所有权限, 并将这些信息存储到 `$_SESSION` 中。之后, 在每次请求需要权限的功能时, 系统不再直接检查用户ID, 而是检查当前用户是否拥有执行该功能所需的权限。例如, 在删除用户之前, 系统会检查 `$_SESSION['permissions']` 中是否包含 `delete_user` 这个权限。

3.4.2 中间件/拦截器

在现代PHP框架 (如Laravel、ThinkPHP) 中, 中间件 (Middleware) 或拦截器 (Interceptor) 是实现统一权限校验的理想工具。中间件提供了一种机制, 可以在HTTP请求到达控制器 (Controller) 之前或之后, 对其进行过滤和处理。通过将权限验证逻辑封装在中间件中, 可以确保所有需要特定权限的路由都经过统一的检查, 避免了在每个控制器方法中重复编写权限验证代码, 从而降低了遗漏的风险。

以Laravel框架为例, 可以创建一个 `CheckAdmin` 中间件:

php

复制

```
// app/Http/Middleware/CheckAdmin.php
namespace App\Http\Middleware;

use Closure;
use Illuminate\Support\Facades\Auth;

class CheckAdmin
{
    public function handle($request, Closure $next)
```

```

{
    // 检查用户是否登录，并且角色是否为admin
    if (!Auth::check() || Auth::user()->role !== 'admin') {
        // 如果不是管理员，则返回403 错误
        abort(403, 'Unauthorized action.');
```

然后，在路由定义中，将这个中间件应用到所有需要管理员权限的路由上：

php

复制

```

// routes/web.php
Route::group(['middleware' => 'admin'], function () {
    Route::get('/admin/dashboard', 'AdminController@dashboard');
    Route::resource('/admin/users', 'AdminController');
    // ... 其他管理员路由
});
```

通过这种方式，任何试图访问 `/admin` 路径下功能的非管理员用户，都会被 `CheckAdmin` 中间件拦截并拒绝，从而有效地防止了垂直越权。

3.4.3 代码示例：修复垂直越权

本节将通过一个具体的PHP代码示例，展示如何修复一个典型的垂直越权漏洞。该示例模拟一个管理员删除用户的功能。

场景描述：

一个Web应用的管理后台提供了删除用户的功能，管理员可以通过访问 `admin/delete_user.php?user_id=xxx` 来删除指定用户。

存在漏洞的代码 (`delete_user_vulnerable.php`)：

php

复制

```

<?php
// delete_user_vulnerable.php
session_start();
```

```
// 只检查了用户是否登录，但没有检查其角色
if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !==
true) {
    header("Location: login.php");
    exit();
}

// 漏洞：任何已登录的用户都可以执行删除操作
$user_id_to_delete = $_GET['user_id'];
$conn = new mysqli("localhost", "db_user", "db_pass", "mydb");
$sql = "DELETE FROM users WHERE id = ?";
$stmt = $conn->prepare($sql);
$stmt->bind_param("i", $user_id_to_delete);
$stmt->execute();

echo "User $user_id_to_delete has been deleted.";
$stmt->close();
$conn->close();
?>
```

漏洞分析：

上述代码只验证了用户是否登录，但没有检查该用户的角色是否为管理员。因此，任何已登录的普通用户，只要知道了这个URL，就可以通过修改 `user_id` 参数来删除任意用户，这是一个典型的垂直越权漏洞。

修复后的代码 (delete_user_secure.php)：

php

复制

```
<?php
// delete_user_secure.php
session_start();

// 1. 检查用户是否登录
if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !==
true) {
    header("Location: login.php");
    exit();
}

// 2. 【核心修复】检查用户角色是否为管理员
if (!isset($_SESSION['role']) || $_SESSION['role'] !== 'admin') {
    // 如果不是管理员，则拒绝访问
}
```

```

        http_response_code(403);
        die("Access Denied: You do not have permission to perform this
action.");
    }

    // 3. 权限验证通过, 安全地执行删除操作
    $user_id_to_delete = $_GET['user_id'];
    $conn = new mysqli("localhost", "db_user", "db_pass", "mydb");
    $sql = "DELETE FROM users WHERE id = ?";
    $stmt = $conn->prepare($sql);
    $stmt->bind_param("i", $user_id_to_delete);
    $stmt->execute();

    if ($stmt->affected_rows > 0) {
        echo "User $user_id_to_delete has been deleted successfully.";
    } else {
        echo "Failed to delete user or user not found.";
    }

    $stmt->close();
    $conn->close();
    ?>

```

修复分析：

修复后的代码在登录检查之后，增加了一个关键的角色权限检查。它从 `$_SESSION` 中获取当前用户的角色，并判断其是否为 `admin`。只有当用户是管理员时，才能继续执行删除操作。对于非管理员用户，程序会返回一个403 Forbidden错误，从而有效地阻止了垂直越权攻击。

这个示例清晰地展示了，通过在服务端增加一道基于角色的权限验证关卡，可以完全修复垂直越权漏洞。这个原则适用于所有需要区分用户权限的功能。

4. 交叉越权 (Hybrid Privilege Escalation)

4.1 漏洞原理

4.1.1 定义与成因

交叉越权 (Hybrid Privilege Escalation)，又称混合越权，是水平越权和垂直越权的结合体。它指的是攻击者在一个攻击请求中，**同时修改了身份标识 (ID) 和权限类型 (Role)**，从而实现了更为复杂的权限跨越。具体来说，攻击者以一个低权限用户（如普通用户）的身份，试图去访问或操作另一个高权限用户（如管理员）的私有数据。这种攻击模式结合了水平

越权（访问他人数据）和垂直越权（跨越权限等级）的特点，因此其攻击路径更加灵活，防御难度也更大。

交叉越权的成因，本质上是水平越权和垂直越权成因的叠加。它通常发生在权限控制逻辑存在多处缺陷的系统中。例如，一个系统可能同时存在以下问题：

- 1. **不安全的直接对象引用（IDOR）**：系统允许用户通过修改URL中的ID参数来访问其他用户的数据（水平越权的基础）。
- 2. **薄弱的角色权限校验**：系统对高权限功能的访问控制不严，或者依赖于客户端传递的、可被篡改的权限标识（垂直越权的基础）。

当这两个条件同时满足时，攻击者就可以构造一个复杂的请求，例如，将请求中的 `user_id` 参数修改为管理员的ID，同时将 `role` 参数从 `user` 修改为 `admin`，从而尝试以普通用户的身份，去访问管理员的数据。

4.1.2 与水平、垂直越权的关系

交叉越权与水平、垂直越权之间存在着紧密的联系和明确的区别。理解它们之间的关系，有助于更全面地认识越权漏洞的本质。

表格 复制			
特征	水平越权	垂直越权	交叉越权
核心操作	修改身份标识（ID）	修改权限类型（Role）	同时修改ID和Role
权限等级	不变（同级别用户间）	改变（低级别到高级别）	改变（低级别到高级别）
攻击目标	同级别其他用户的数据	高权限用户的功能/数据	高权限用户的数据
攻击起点	普通用户A	普通用户A	普通用户A
攻击目标	普通用户B的数据	管理员的功能/数据	管理员B的数据
复杂性	相对较低	中等	最高
防御难度	相对较低	中等	最高

关系总结：

- **水平越权是基础**：许多交叉越权攻击的第一步，就是先利用水平越权的思路，找到系统中存在的IDOR漏洞，确定可以通过修改ID来访问其他用户的数据。

- **垂直越权是提升**：在找到IDOR漏洞的基础上，攻击者会进一步尝试提升自己的权限等级，以访问更高权限用户的数据。这通常通过修改角色参数或利用其他垂直越权漏洞来实现。
- **交叉越权是组合**：它将两种攻击手法结合起来，形成了一种“1+1>2”的攻击效果。攻击者不再是简单地访问同级别用户的数据，也不是简单地获得管理员功能，而是**以一个低权限用户的身份，精准地攻击一个高权限用户的数据**，这在某些场景下可能具有更高的价值（例如，窃取管理员的敏感信息）。

4.2 漏洞挖掘方法

4.2.1 结合水平与垂直越权的挖掘方法

挖掘交叉越权漏洞，需要综合运用挖掘水平越权和垂直越权的方法。其核心思路是：**在发现系统存在IDOR（水平越权）漏洞的基础上，进一步测试该系统是否存在垂直越权漏洞，并尝试将两者结合起来进行攻击。**

具体的挖掘步骤如下：

1. **发现水平越权漏洞**：首先，按照第二章介绍的方法，对系统进行全面的水平越权测试。通过修改请求中的ID参数，尝试访问其他同级别用户的数据。如果发现系统存在IDOR漏洞，例如，用户A可以通过修改 `user_id` 参数来查看用户B的资料，那么我们就找到了交叉越权攻击的第一个“支点”。
2. **测试垂直越权漏洞**：在确认系统存在水平越权漏洞后，接下来需要测试该系统是否存在垂直越权漏洞。按照第三章介绍的方法，尝试以一个低权限用户的身份，去访问高权限用户的功能或数据。例如，尝试修改Cookie中的 `role` 参数，或者直接访问管理员功能的URL。
3. **构造交叉越权攻击**：如果系统同时存在水平越权和垂直越权的漏洞，那么就可以构造交叉越权攻击了。攻击者可以先通过水平越权的方法，获取到系统中其他用户的ID，特别是管理员的ID。然后，在发起攻击请求时，**同时修改两个关键参数**：将身份标识（ID）修改为管理员的ID，将权限类型（Role）修改为 `admin`。
4. **验证攻击结果**：发送构造好的交叉越权请求，观察服务器的响应。如果服务器返回了管理员用户的私有数据，那么就成功地发现了一个交叉越权漏洞。

4.3 漏洞挖掘技巧

4.3.1 识别权限类型与ID的变化

在挖掘交叉越权漏洞时，一个关键的技巧是**识别并区分请求中代表“权限类型”和“身份标识”的参数**。这需要测试人员对HTTP请求进行细致的分析。

识别权限类型参数：

- **参数名：**通常与角色或权限相关，如 `role` , `type` , `privilege` , `is_admin` , `user_level` 等。
- **参数值：**通常是代表不同权限等级的字符串或数字，如 `user` , `admin` , `1` , `2` 。
- **出现位置：**可能出现在URL参数、POST请求体、Cookie或HTTP头中。

识别身份标识参数：

- **参数名：**通常与ID或用户名相关，如 `id` , `user_id` , `uid` , `username` 等。
- **参数值：**通常是代表特定用户的唯一标识符，如数字ID、用户名、邮箱等。
- **出现位置：**同样可能出现在URL参数、POST请求体、Cookie或HTTP头中。

实战技巧：

1. **对比不同权限用户的请求：**准备两个账户，一个普通用户和一个管理员。分别登录这两个账户，并执行相同的操作（如果管理员有权限而普通用户没有）。使用Burp Suite捕获这两个请求，并进行对比。通过对比，可以清晰地看到哪些参数代表了权限差异，哪些参数代表了身份差异。
2. **关注多步骤操作的流程：**在某些复杂的业务流程中，权限和ID可能在不同的步骤中传递。例如，第一步登录，返回一个包含角色信息的Cookie；第二步访问某个功能，URL中包含用户ID。在这种情况下，需要对整个流程进行跟踪，找到所有可以修改的参数。
3. **利用Burp Suite的Intruder进行组合攻击：**Burp Suite的Intruder模块支持多种攻击类型，如“Cluster bomb”，它可以同时对多个payload位置进行组合攻击。可以将权限类型参数和身份标识参数都设置为payload位置，然后加载不同的字典，让Intruder自动生成所有可能的组合，从而高效地发现交叉越权漏洞。

4.4 整改措施

4.4.1 综合水平与垂直越权的修复方法

修复交叉越权漏洞，需要同时解决水平越权和垂直越权的问题。其核心思想是：**在服务器端对每一次数据访问请求，都进行严格的、双重维度的权限验证。**

1. **强制角色权限校验（解决垂直越权）：**在处理任何需要权限的功能时，首先必须验证当前登录用户的角色是否满足要求。这可以通过RBAC模型或中间件来实现。确保低权限用户无法访问任何高权限功能。

2. **强制资源归属校验（解决水平越权）**：在角色权限校验通过之后，还需要验证当前用户是否有权访问或操作请求中指定的数据对象。这需要在数据访问层进行，通过关联查询来确保数据的所有权。
3. **不信任任何客户端数据**：无论是身份标识（ID）还是权限类型（Role），都不能信任客户端传来的值。当前用户的ID和角色信息，必须只从服务器端的会话（`$_SESSION`）中获取。请求中携带的ID参数，只能作为要操作的目标，而不能作为判断依据。

通过将这两种修复方法结合起来，可以构建一个坚固的、双重保险的权限控制系统，从而有效地防止交叉越权攻击。

4.4.2 代码示例：修复交叉越权

本节将通过一个PHP代码示例，展示如何修复一个典型的交叉越权漏洞。该示例模拟一个普通用户试图访问并修改管理员个人资料的场景。

场景描述：

一个Web应用的个人资料修改功能，其API接口为 `POST /api/profile/update`。请求体中包含 `user_id`（要修改的用户ID）和 `role`（发起请求者的角色，用于权限判断）两个参数。

存在漏洞的代码（`update_profile_vulnerable.php`）：

php

复制

```
<?php
// update_profile_vulnerable.php - API接口
session_start();

// 只检查了是否登录
if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {
    http_response_code(401);
    die(json_encode(['error' => 'Unauthorized']));
}

// 获取请求参数
$data = json_decode(file_get_contents('php://input'), true);
$target_user_id = $data['user_id'];
$requester_role = $data['role']; // 漏洞1：信任客户端传来的角色
$new_email = $data['email'];

// 漏洞2：只根据角色判断，没有验证资源归属
```

```

if ($requester_role === 'admin') {
    // 执行更新操作
    $conn = new mysqli("localhost", "db_user", "db_pass", "mydb");
    $sql = "UPDATE users SET email = ? WHERE id = ?";
    $stmt = $conn->prepare($sql);
    $stmt->bind_param("si", $new_email, $target_user_id);
    $stmt->execute();
    echo json_encode(['success' => 'Profile updated.']);
} else {
    http_response_code(403);
    die(json_encode(['error' => 'Permission denied']));
}
?>

```

漏洞分析：

上述代码存在两个关键漏洞：

1. **垂直越权**：它信任了客户端传来的 `role` 参数，攻击者可以将 `role=user` 修改为 `role=admin` 来伪造管理员身份。
2. **水平越权**：它没有验证 `target_user_id` 是否属于当前登录用户，任何知道用户ID的攻击者都可以尝试修改他人的资料。

这两个漏洞结合起来，就构成了一个典型的交叉越权漏洞：一个普通用户可以通过修改 `role` 和 `user_id` 参数，来修改管理员的资料。

修复后的代码 (update_profile_secure.php)：

php

复制

```

<?php
// update_profile_secure.php - API接口
session_start();

// 1. 检查是否登录
if (!isset($_SESSION['logged_in']) || $_SESSION['logged_in'] !== true) {
    http_response_code(401);
    die(json_encode(['error' => 'Unauthorized']));
}

// 2. 从会话中获取当前登录用户的ID和角色（可信）
$current_user_id = $_SESSION['user_id'];
$current_user_role = $_SESSION['role'];

```

```

// 3. 获取请求参数 (不可信)
$data = json_decode(file_get_contents('php://input'), true);
$target_user_id = $data['user_id'];
$new_email = $data['email'];

// 4. 【核心修复】双重权限验证
$is_authorized = false;

// 4.1 验证资源归属：用户只能修改自己的资料
if ($target_user_id == $current_user_id) {
    $is_authorized = true;
}

// 4.2 验证角色权限：管理员可以修改任何人的资料
elseif ($current_user_role === 'admin') {
    $is_authorized = true;
}

if ($is_authorized) {
    // 执行更新操作
    $conn = new mysqli("localhost", "db_user", "db_pass", "mydb");
    $sql = "UPDATE users SET email = ? WHERE id = ?";
    $stmt = $conn->prepare($sql);
    $stmt->bind_param("si", $new_email, $target_user_id);
    $stmt->execute();
    echo json_encode(['success' => 'Profile updated.']);
} else {
    http_response_code(403);
    die(json_encode(['error' => 'Permission denied: You can only
update your own profile.']));
}
?>

```

修复分析：

修复后的代码引入了双重权限验证逻辑：

1. **资源归属校验**：首先检查 `target_user_id` 是否与 `$current_user_id` 一致。如果一致，说明用户在修改自己的资料，授权通过。
2. **角色权限校验**：如果用户不是修改自己的资料，那么系统会检查其角色是否为 `admin` 。如果是管理员，则授权通过。
通过这种“或”逻辑，系统同时解决了水平越权（用户不能修改他人资料）和垂直越权（只有管理员能修改他人资料）的问题，从而有效地修复了交叉越权漏洞。

5. 结论与建议

5.1 总结

本报告对Web应用中的越权漏洞进行了系统性的研究，深入剖析了其三种主要形式——水平越权、垂直越权和交叉越权的原理、挖掘方法、技巧及整改措施，并特别结合了PHP编程语言的特性进行了阐述。

核心观点总结如下：

- 越权漏洞的本质是“失效的访问控制”：其根本原因在于应用程序未能对用户请求的权限进行严格、有效的服务器端验证，过分信任了来自客户端的数据。
- 水平越权的核心是“数据所有权”校验的缺失：攻击者通过修改身份标识（ID），访问同级别其他用户的数据。修复的关键在于后端强制进行资源归属验证。
- 垂直越权的核心是“功能/角色权限”校验的缺失：攻击者通过提升权限等级，执行高权限用户的功能。修复的关键在于实施严格的角色权限控制，如RBAC模型。
- 交叉越权是前两者的结合，危害更大：它要求防御体系同时解决数据所有权和角色权限两个维度的校验问题。
- “默认拒绝”和“最小权限”是核心防御原则：系统应默认拒绝所有访问请求，只授予用户完成其工作所必需的最小权限集合。

5.2 安全开发建议

为了从源头上预防越权漏洞的产生，开发团队应将安全理念贯穿于整个软件开发生命周期（SDLC）中。以下是一些面向开发人员和安全团队的具体建议：

表格

复制

阶段	安全建议	具体实践
设计阶段	实施威胁建模	在设计初期，识别系统中所有涉及权限控制的功能点，并设计相应的安全控制措施。
	采用安全的架构模式	优先采用成熟的权限控制模型，如RBAC或ABAC。将逻辑解耦，实现集中、统一的权限控制。
编码阶段	遵循安全编码规范	永远不要信任客户端数据。所有权限验证必须在服务器端进行，必须加入基于当前用户身份的关联查询。
	使用参数化查询	在构造SQL语句时，务必使用预处理语句（PreparedStatement）或ORM框架提供的参数绑定功能，以防止SQL注入漏洞。
	避免不安全的直接对象引用	尽量避免在URL或API中暴露数据库主键等内部标识符。对其进行加密、哈希或使用随机生成的、不可预测的ID。
测试阶段	将越权测试纳入安全测试流程	在功能测试和渗透测试中，应包含针对水平、垂直和交叉权限的测试用例。使用Burp Suite等工具进行系统性的手动和自动化测试。
	进行代码安全审计	在代码合并（Merge Request）前，进行严格的代码审查，重点关注所有涉及权限判断和数据操作的代码逻辑。
部署与运维	实施纵深防御	在应用层之外，部署Web应用防火墙（WAF）、入侵检测系统（IDS）等安全设备。虽然它们对逻辑漏洞的检测能力有限，但可以结合规则库识别和已知的攻击模式。
	启用安全日志与监控	对所有敏感操作（如登录、权限变更、数据删除）进行详细记录。建立安全事件监控和告警机制，及时发现和响应可疑的越权行为。

5.3 未来研究方向

尽管越权漏洞的研究已经相当深入，但随着Web技术的不断发展，新的攻击面和防御技术也在不断涌现。未来的研究方向可以集中在以下几个方面：

- 自动化检测技术的改进：**目前，越权漏洞的检测在很大程度上依赖于人工测试。未来的研究可以探索如何利用机器学习、程序分析等技术，开发出能够更智能地理解业务逻辑、自动生成测试用例的自动化检测工具，以提高漏洞发现的效率和覆盖率。
- 针对API和微服务架构的越权研究：**随着前后端分离和微服务架构的普及，API成为了主要的攻击目标。API的权限控制模型与传统Web应用有所不同，其无状态性和复杂的调用链

给越权防御带来了新的挑战。未来的研究可以针对API网关、服务间通信（如JWT令牌的使用）等场景，提出更精细化的越权防御方案。

3. **基于属性的访问控制（ABAC）的实践与应用：**相比于RBAC，ABAC提供了更细粒度、更灵活的权限控制能力。它可以根据用户、资源、环境等多种属性动态地做出授权决策。未来的研究可以探索如何在复杂的业务系统中高效地实现和管理ABAC模型，并将其应用于越权漏洞的防御。
4. **DevSecOps中的权限控制集成：**将安全实践深度集成到开发和运维流程中是未来的趋势。研究如何在CI/CD流水线中自动进行权限控制规则的验证、部署和监控，实现权限管理的自动化和智能化，将是保障大规模分布式系统安全的关键。

总之，越权漏洞的攻防是一个持续演进的过程。只有不断地研究新的攻击技术，并开发出更先进的防御手段，才能在日益复杂的网络环境中，有效地保障Web应用和数据的安全。